

All for one and none for all

Compiling polymorphic relations without monomorphization

Dmitri Volkov, Yafei Yang, Chung-chieh Shan

2026 August 24

Introducing semiringKanren

Goals

Types

Computing relations

Polymorphic semiringKanren

Sum swap

Expanding sum swap

Compilation

semiringKanren represents relations as finite multidimensional arrays.

Polymorphic relations need to represent arbitrarily many specifically-typed relations.

How to do both?

All for one and none for all

Represent all concrete
instances of a
polymorphic relation

using only a single
concrete instance

to compile away type
variables

Introducing semiringKanren

Goals

Types

Computing relations

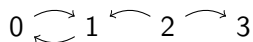
Polymorphic semiringKanren

Sum swap

Expanding sum swap

Compilation

Representing relations



```

(defrel (graph
  (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
  
```

	to			
	0	1	2	3
from	0	0	1	0
	1	1	0	0
	2	0	1	0
	3	0	0	0

Represent graphs as relations or as adjacency matrices.
semiringKanren computes matrices for arbitrary relations!

Primitive relations

$(==\ x\ y)$

		y			
		0	1	2	3
x	0	1	0	0	0
	1	0	1	0	0
	2	0	0	1	0
	3	0	0	0	1

$(\neq\ x\ y)$

		y			
		0	1	2	3
x	0	0	1	1	1
	1	1	0	1	1
	2	1	1	0	1
	3	1	1	1	0

disj is addition

```
(defrel (r0 (x : (Num 4)) (y : (Num 4)))
  (disj
    (== x 0)
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

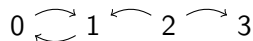
conj is multiplication

```
(defrel (r1 (x : (Num 4)) (y : (Num 4)))
  (disj
    (== x 0)
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

fresh is summation

Vertices with incoming edges.



```

(defrel (incoming-edge-vertices (y : (Num 4)))
  (fresh (x : (Num 4))
    (graph x y)))
  
```

$$\sum_x \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 & 1 \end{bmatrix}$$

Types

semiringKanren values inhabit finite algebraic data types.

	left	right
(left 0) : (Sum (Num 2) (Num 3))	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$
(left 1) : (Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$
(right 0) : (Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$
(right 1) : (Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$
(right 2) : (Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$

(semiringKanren also supports unit and product types)

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3)))))
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$



Introducing semiringKanren

Goals

Types

Computing relations

Polymorphic semiringKanren

Sum swap

Expanding sum swap

Compilation

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

0	0	0	1	0	0
0	0	0	0	1	0
0	0	0	0	0	1
1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0

 $x \mapsto (\text{left } 0)$ $y \mapsto (\text{right } 0)$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } 2)$

$y \mapsto (\text{left } 2)$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

0	0	0	1	0	0
0	0	0	0	1	0
0	0	0	0	0	1
1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

0	0	0	1	0	0
0	0	0	0	1	0
0	0	0	0	0	1
1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

0	0	0	0	1	0	0
0	0	0	0	0	1	0
0	0	0	0	0	0	1
1	0	0	0	0	0	0
0	1	0	0	0	0	0
0	0	1	0	0	0	0
0	0	0	1	0	0	0

 $x \mapsto (\text{left } 0)$ $y \mapsto (\text{right } 0)$

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

0	0	0	0	1	0	0
0	0	0	0	0	1	0
0	0	0	0	0	0	1
1	0	0	0	0	0	0
0	1	0	0	0	0	0
0	0	1	0	0	0	0
0	0	0	1	0	0	0

$x \mapsto (\text{right } 3)$

$y \mapsto (\text{left } 3)$

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

All for one and none for all

Represent all concrete
instances of a
polymorphic relation

using only a single
concrete instance

to compile away type
variables

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$x \mapsto (\text{left } _ \alpha)$

$y \mapsto (\text{left } _ \beta)$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$$x \mapsto (\text{right } _ \beta)$$

$$y \mapsto (\text{right } _ \alpha)$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Succeeds for:

$$\left. \begin{array}{l} x \mapsto (\text{left } \underline{0}) \\ y \mapsto (\text{right } \underline{0}) \end{array} \right\} \alpha\text{-typed parts equal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Succeeds for:

$$\left. \begin{array}{l} x \mapsto (\text{left } _ \alpha) \\ y \mapsto (\text{right } _ \alpha) \end{array} \right\} \alpha\text{-typed parts equal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Fails for:

$$\left. \begin{array}{l} x \mapsto (\text{left } \underline{0}) \\ y \mapsto (\text{right } \underline{2}) \end{array} \right\} \alpha\text{-typed parts disequal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$$\left. \begin{array}{l} x \mapsto (\text{left } _ \alpha) \\ y \mapsto (\text{right } _ \alpha) \end{array} \right\} \alpha\text{-typed parts disequal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Succeeds for:

$$\left. \begin{array}{l} x \mapsto (\text{right } _ \beta) \\ y \mapsto (\text{left } _ \beta) \end{array} \right\} \beta\text{-typed parts equal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$$\left. \begin{array}{l} x \mapsto (\text{right } \neg\beta) \\ y \mapsto (\text{left } \neg\beta) \end{array} \right\} \beta\text{-typed parts disequal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Same success/failure results when:

- ▶ Non-variable typed parts are the same
- ▶ Variable-typed parts have the same (dis)equalities

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{left } \underline{2}_\beta)$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{left } \underline{3}_\beta)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{left } \underline{2}_\beta)$

x is (right ...)

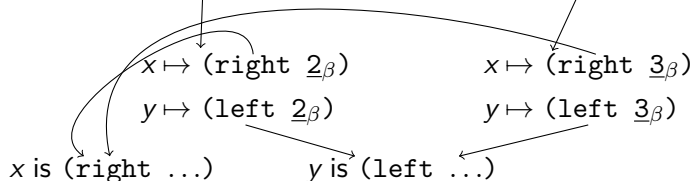
$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{left } \underline{3}_\beta)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

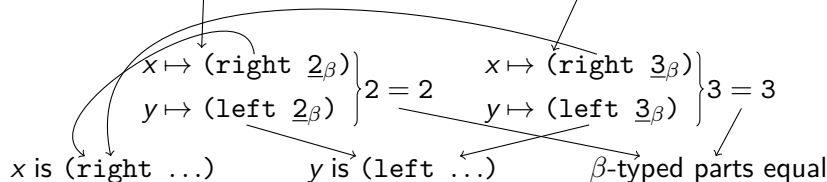
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$



Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$



Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{left } \underline{1}_\beta)$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{left } \underline{2}_\beta)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{left } \underline{1}_\beta)$

x is (right ...)

$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{left } \underline{2}_\beta)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

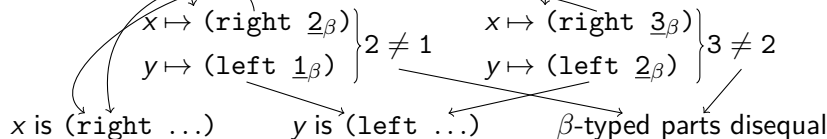
$x \mapsto (\text{right } \underline{2}_\beta)$
 $y \mapsto (\text{left } \underline{1}_\beta)$
 $x \text{ is } (\text{right } \dots)$
 $y \text{ is } (\text{left } \dots)$

$x \mapsto (\text{right } \underline{3}_\beta)$
 $y \mapsto (\text{left } \underline{2}_\beta)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$



Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{right } \underline{0}_\alpha)$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{right } \underline{0}_\alpha)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{right } \underline{0}_\alpha)$

x is (right ...)

$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{right } \underline{0}_\alpha)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\text{right } \underline{2}_\beta)$

$y \mapsto (\text{right } \underline{0}_\alpha)$

x is (right ...)

y is (right ...)

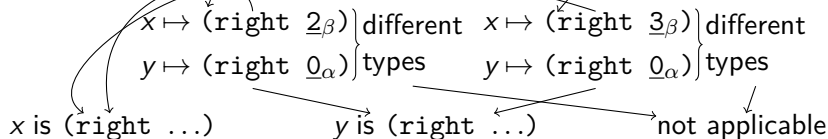
$x \mapsto (\text{right } \underline{3}_\beta)$

$y \mapsto (\text{right } \underline{0}_\alpha)$

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$



Has disequality

```
(defrel (has-disequality (y :  $\alpha$ ))  
  (fresh (x :  $\alpha$ )  
    (=/= x y)))
```

Has disequality

```
(defrel (has-disequality (y :  $\alpha$ ))  
  (fresh (x :  $\alpha$ )  
    (=/= x y)))
```

$$\alpha = \begin{bmatrix} \text{Num } 2 \\ 1 & 1 \end{bmatrix}$$
$$\alpha = \begin{bmatrix} \text{Num } 1 \\ 0 \end{bmatrix}$$

Has disequality

```
(defrel (has-disequality (y :  $\alpha$ ))  
  (fresh (x :  $\alpha$ )  
    (=/= x y)))
```

$\alpha = (\text{Num } 2)$

$\begin{bmatrix} 1 & 1 \end{bmatrix}$

Always succeeds

$\alpha = (\text{Num } 1)$

$\begin{bmatrix} 0 \end{bmatrix}$

Always fails

Has disequality

```
(defrel (has-disequality (y :  $\alpha$ ))  
  (fresh (x :  $\alpha$ )  
    (=/= x y)))
```

$\alpha = (\text{Num } 2)$

$\begin{bmatrix} 1 & 1 \end{bmatrix}$

Always succeeds

$\alpha = (\text{Num } 1)$

$\begin{bmatrix} 0 \end{bmatrix}$

Always fails

Expansion does not work here.

Has disequality

```
(defrel (has-disequality (y :  $\alpha$ ))  
  (fresh (x :  $\alpha$ )  
    (=/= x y)))
```

$\alpha = (\text{Num } 2)$
[1 1]

Always succeeds

$\alpha = (\text{Num } 1)$
[0]

Always fails

Expansion does not work here.

Need to handle small-sized types separately (not really “all for one”).

Compilation

1. Calculate type sizes

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
 - ▶ Large-enough calls all use the same relation

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
 - ▶ Large-enough calls all use the same relation
3. Expand monomorphic relation calls back to the right types
 - ▶ By generating vanilla semiringKanren code

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
 - ▶ Large-enough calls all use the same relation
3. Expand monomorphic relation calls back to the right types
 - ▶ By generating vanilla semiringKanren code

More details in the paper!

Introducing semiringKanren

Goals

Types

Computing relations

Polymorphic semiringKanren

Sum swap

Expanding sum swap

Compilation

Conclusion

Represent the (dis)equality structure of relations compactly, and expand as needed.

Conclusion

Represent the (dis)equality structure of relations compactly, and expand as needed.

Works because $==$ and $\neq$ are the only primitive relations.

Conclusion

Represent the (dis)equality structure of relations compactly, and expand as needed.

Works because $==$ and $\neq$ are the only primitive relations.

Does this generalize?